Hintergrund-Abgleich

Lernunterlage: BackgroundService, PeriodicTimer und ein Kontingent, das reichen muss

1 Worum es geht

Bis zum 02.11.2026 hat Home Control Tuya nur gefragt, wenn jemand auf **„Mit Tuya abgleichen“** geklickt hat. An diesem Tag wurde die Tür geöffnet, Tuya meldete `doorcontact_state = true`, die App zeigte aber weiter **ZU**. Das Ereignis `door_opened` fehlte im Verlauf.

Lösung: Das Backend fragt selbst regelmäßig nach, auch wenn niemand die App offen hat. Das nennt man **Polling** (regelmäßig nachfragen).

2 Zuerst rechnen: das Kontingent

Das Tuya-Cloud-Projekt hat ein Kontingent an API-Aufrufen pro Monat. Die Produktseite von Tuya IoT Core nennt für die Testversion **26 000 Aufrufe/Monat**. Nachsehen: platform.tuya.com → Cloud → My Services → IoT Core. Bevor ein Takt festgelegt wurde, wurde deshalb gerechnet.

Ein Durchlauf kostet **1 Status-Aufruf pro Gerät** (hier 4) plus **1 Aufruf der Geräteliste** (online/offline) = 5 Aufrufe. Dazu etwa alle 2 Stunden ein neues Token.

Takt	Durchläufe/Tag	Aufrufe/Tag	Aufrufe/Monat	bei 26 000
1 min	1 440	7 200	216 000	8-fach zu viel
2 min	720	3 600	108 000	4-fach zu viel
5 min	288	1 440	43 200	zu viel
10 min	144	720	21 600	kaum Puffer
15 min (erste Version)	96	480	14 400	~11 000 Puffer

```
Rechenweg für 15 min:
  Durchläufe pro Tag = 24 h × 60 min ÷ 15 min = 96
  Aufrufe pro Tag    = 96 × (4 Geräte + 1 Liste) = 480
  Aufrufe pro Monat  = 480 × 30 = 14 400
```

Nachtrag 02.11. abends: Takt pro Gerät

Die Diagnose zeigte: Wichtig ist vor allem die **Tür**. Deshalb hat jedes Gerät jetzt einen eigenen Takt (`Device.TuyaSyncSeconds`, auf der Geräteseite unter „Abgleich“). Die Tür wird alle **3 Minuten** gelesen, alle anderen im Standard-Takt von **30 Minuten**. Der Dienst wacht jede Minute auf und fragt nur die fälligen Geräte; die Online-Liste nur alle 30 Minuten.

Lauf	Läufe/Tag	Aufrufe je Lauf	Aufrufe/Monat
alle 30 min: 4 Geräte + Online-Liste	48	5	7 200
dazwischen alle 3 min: nur Tür	432	1	12 960
Token			~360

Lauf	Läufe/Tag	Aufrufe je Lauf	Aufrufe/Monat
Summe			~20 500

```
Rechenweg:
  Tür-Läufe pro Tag   = 24 × 60 ÷ 3 = 480, davon 48 zusammen mit den anderen
  nur Tür             = 480 − 48 = 432 Läufe × 1 Aufruf  = 432
  alle 30 min         = 48 Läufe × (4 Geräte + 1 Liste)  = 240
  pro Tag             = 672, pro Monat = 672 × 30 = 20 160 (+ ~360 Token)
```

Die versteckten Kosten: Status-LEDs

Die Status-LEDs fragen jede Minute GET `/api/status`, und der fragte bisher (mit 60-s-Cache) die Tuya-Geräteliste ab. Ist die App irgendwo dauerhaft offen (Tablet an der Wand), sind das **1 440 Aufrufe/Tag = 43 200/Monat**, nur für die LEDs. Lösung: Der Hintergrund-Abgleich füllt den Speicher (`TuyaOnlineCache`), und `/api/status` liest ihn nur noch.

Variante (App 24/7 offen)	LEDs	Abgleich	Summe/Monat
Nichts geändert	43 200	11 520	~55 000
Status-Cache 5 min	8 640	11 520–14 400	~20 500–23 400
Abgleich füllt Cache (gewählt)	0	14 400	~14 800

> Erst rechnen, dann programmieren: Ein Takt, der in der Entwicklung harmlos wirkt, kann im Dauerbetrieb ein Monatskontingent in wenigen Tagen aufbrauchen. Und an ALLE Aufrufer denken, nicht nur an den neuen.

3 BackgroundService und PeriodicTimer

Ein **BackgroundService** ist Code, der neben den HTTP-Anfragen dauerhaft im Backend läuft. Er startet mit der App und wird beim Beenden gestoppt. Registriert wird er mit `AddHostedService`.

```
builder.Services.AddHostedService<TuyaSyncBackgroundService>();

protected override async Task ExecuteAsync(CancellationToken stoppingToken)
{
    var interval = options.CurrentValue.GetAutoSyncInterval();   // null = aus
    if (interval is null) return;

    using var timer = new PeriodicTimer(interval.Value, timeProvider);
    do
    {
        failures = await RunOnceAsync(stoppingToken) ? 0 : failures + 1;
        timer.Period = GetDelay(interval.Value, failures);       // Backoff
    }
    while (await timer.WaitForNextTickAsync(stoppingToken));
}
```

- **PeriodicTimer:** „Warte bis zum nächsten Takt“ als `await`, ohne Callback. Läuft ein Durchlauf länger als der Takt, gibt es keine Überschneidung.
- **do … while:** Der erste Durchlauf kommt sofort beim Start, damit Änderungen während eines Neustarts gleich übernommen werden.
- **stoppingToken:** Wird beim Beenden der App ausgelöst. `WaitForNextTickAsync` endet dann, der Dienst hört sauber auf.
- **TimeProvider:** Die Uhr kommt von außen. Im Test stellt `FakeTimeProvider.Advance` die Zeit vor, statt 15 Minuten zu warten.

4 Scopes im Singleton

Ein BackgroundService ist ein **Singleton**: Es gibt ihn einmal, solange die App läuft. `TuyaDeviceSyncer` und `ApplicationDbContext` sind dagegen **scoped**: Sie leben eine HTTP-Anfrage lang. Im Hintergrund gibt es keine Anfrage, also baut der Dienst sich pro Durchlauf selbst einen Scope:

```
using var scope = scopeFactory.CreateScope();
var syncer = scope.ServiceProvider.GetRequiredService<TuyaDeviceSyncer>();
var results = await syncer.ReadAllAsync(apply: true, cancellationToken);
// Ende von using: DbContext wird freigegeben
```

Lebensdauer	Beispiel	gilt
Singleton	TuyaSyncBackgroundService, TuyaOnlineCache, TuyaTokenStore	solange die App läuft
Scoped	ApplicationDbContext, TuyaDeviceSyncer	eine Anfrage bzw. ein Scope
Transient	TuyaClient (typisierter HttpClient)	bei jedem Abruf neu

> Einen scoped Dienst nie direkt in einen Singleton geben (Captive Dependency): Der DbContext lebte dann ewig, sammelte alle Objekte an und wäre nicht threadsicher. Stattdessen IServiceScopeFactory und pro Durchlauf ein eigener Scope.

Knopf und Hintergrund rufen dieselbe Methode auf. Die vorhandene Sperre (`SemaphoreSlim SyncLock`) lässt nur einen Abgleich gleichzeitig zu, sonst könnte dasselbe Ereignis doppelt gespeichert werden.

5 Fehler und Backoff

Wirft ein BackgroundService eine Exception, beendet .NET standardmäßig die **ganze App**. Deshalb fängt `RunOnceAsync` jeden Fehler ab, schreibt Typ und Meldung ins Log (nie Zugangsdaten) und versucht es beim nächsten Takt erneut. Nur das Beenden der App wird durchgereicht.

Backoff: Ist die Tuya-Cloud gestört, wäre es sinnlos und unhöflich, sie im selben Takt weiter zu fragen. Nach jedem Fehler in Folge verdoppelt sich der Abstand, höchstens bis 1 Stunde:

```
Fehler in Folge:   0      1      2      3      4 …
Prüfabstand:       1 min  2 min  4 min  8 min  16 min … (Obergrenze 1 h)
Erster Erfolg  → wieder jede Minute prüfen
```

Das nennt man **exponentiellen Backoff** (Abstand × 2 pro Fehler). Ohne Zugangsdaten gilt das nicht als Fehler: Tuya ist dann einfach nicht eingerichtet, und es wird gar nicht gefragt.

6 Warum Polling und nicht Push?

	Polling (gewählt)	Push (Tuya Message Service)
Wie	Backend fragt nach (Tür alle 3, Rest alle 30 min)	Tuya schickt jede Änderung sofort über eine dauerhafte Verbindung
Zeitpunkt	bis zu 3 bzw. 30 min zu spät	genau
Kurze Änderungen	Tür kurz auf und zu: nicht gesehen	jede Änderung kommt an

	Polling (gewählt)	Push (Tuya Message Service)
Aufwand	ein BackgroundService, vorhandener Code	eigene Bibliothek, dauerhafte Verbindung, Entschlüsselung, Wiederverbinden
Kontingent	API-Aufrufe (hier 14 400/Monat)	eigenes Nachrichten-Kontingent
Erklärbar	in zwei Sätzen	eigenes Thema

Für ein Umschulungsprojekt mit 4 Geräten ist Polling der richtige erste Schritt: wenig Code, gut testbar, gut erklärbar. Push wäre der Weg, wenn genaue Zeitpunkte wichtig werden (z. B. Alarm bei offener Tür).

7 Was die Oberfläche macht

Übersicht und Geräteseite laden die Geräteliste jede Minute und beim Zurückkehren in die App neu (Hook `useAutoRefresh`, Muster wie die Status-LEDs). Das kostet **kein** Tuya-Kontingent, denn gelesen wird nur die eigene Datenbank.

```
const timer = window.setInterval(() => latest.current(), 60_000)
document.addEventListener('visibilitychange', onVisible)   // iPhone entsperrt → sofort
```

8 Grenzen

- **Bis zu 3 (Tür) bzw. 30 Minuten Verzögerung.** Das Ereignis bekommt den Zeitpunkt des Abrufs, nicht den der echten Änderung.
- **Kurze Änderungen gehen verloren.** Tür auf und nach 2 Minuten wieder zu: Beide Abrufe sehen „ZU“.
- **Online/Offline bis zu 30 Minuten alt**, weil die LEDs die Online-Liste des letzten 30-Minuten-Laufs lesen.
- **Jedes Gerät kostet.** Ein weiteres Gerät im 30-Minuten-Takt +1 440/Monat, im 3-Minuten-Takt +14 400.
- **Takt ändern erfordert einen Neustart** des Dienstes.
- **Eine Backend-Instanz.** Bei mehreren Instanzen würde jede selbst abgleichen (doppeltes Kontingent). Deshalb ist der Hintergrund-Abgleich beim Entwickeln aus (`Tuya:AutoSyncSeconds = 0` in appsettings.Development.json): Dienst und app-starten.sh teilen sich dieselbe Datenbank. Einschalten nur bei gestopptem Dienst.

9 Tests

Test	prüft
Durchlauf_ruft_den_Abgleich_auf_…	Tür OFFEN gespeichert, Ereignis door_opened, Cache gefüllt
Ohne_Zugangsdaten_wird_Tuya_nicht_gefragt	kein Aufruf, kein Fehler
Tuya_nicht_erreichbar_wird_geloggt_statt_geworfen	Warnung im Log, kein Secret im Log
Unerwarteter_Fehler_wird_geloggt_statt_geworfen	Fehler im Log, Dienst läuft weiter
Backoff_verdoppelt_bis_hoechstens_eine_Stunde	60 → 120 → 240 … → 3600 s
Null_heisst_aus	AutoSyncSeconds 0: Dienst endet sofort
Laeuft_im_Takt_wird_bei_Fehlern_langsamer_…	Takt, Backoff, sauberes Stoppen mit Test-Uhr

Test	prüft
StatusCacheModeTests (3)	10 × /api/status: 0 Tuya-Aufrufe mit Abgleich, 1 ohne

Die Testfabrik schaltet den Hintergrund-Abgleich fest aus (`Tuya:AutoSyncSeconds = 0`). Sonst würde er beim Start der Test-API mit den echten User Secrets die echte Tuya-Cloud fragen.

10 Kurzantworten für Rückfragen

Frage	Antwort
Was ist ein BackgroundService?	Code, der dauerhaft neben den HTTP-Anfragen im Backend läuft, mit der App startet und stoppt.
Warum PeriodicTimer statt Task.Delay?	Fester Takt, als await lesbar, und mit TimeProvider testbar.
Warum ein eigener Scope?	Der Dienst ist Singleton, DbContext und Syncer sind scoped.
Warum Tür 3, Rest 30 Minuten?	Die Tür ist am wichtigsten. Zusammen ~20 500 von 26 000 Aufrufen/Monat.
Was ist Backoff?	Bei Fehlern seltener fragen (× 2, höchstens 1 h), damit eine gestörte Cloud nicht geflutet wird.
Warum stürzt der Dienst nie ab?	Jeder Fehler wird gefangen und geloggt; eine Exception würde sonst die ganze App beenden.
Warum kein Push?	Mehr Aufwand (dauerhafte Verbindung, eigene Bibliothek). Polling reicht für den Anfang.
Kosten die LEDs noch Kontingent?	Nein, sie lesen den Speicher, den der Abgleich füllt.

11 Merksätze

- Erst das Kontingent rechnen, dann den Takt festlegen, und dabei an alle Aufrufer denken.
- BackgroundService + PeriodicTimer = regelmäßige Arbeit im Backend.
- Scoped Dienste im Singleton nur über IServiceScopeFactory, ein Scope pro Durchlauf.
- Ein Hintergrund-Dienst darf nie abstürzen: Fehler fangen, loggen, Backoff.
- Polling ist einfach, aber verzögert und sieht kurze Änderungen nicht. Push ist genau, aber aufwendig.